

Smart Expense Tracker & Budget Prediction System

COMPLETE DAY-WISE PROJECT DEVELOPMENT LIFECYCLE DOCUMENTATION

Repository Target Profile

This document details the step-by-step technical blueprints, system requirements, schemas, and design wireframes mapped out for your GitHub repository implementation architecture.

Day 01: Project Title & Architecture Profile

Project Title: Smart Expense Tracker & Budget Prediction System

Core Objective: To build an intelligent personal financial platform that empowers users to effortlessly log routine transactions, categorize structural expenses, dynamically visualize spending velocity, and leverage mathematical machine learning metrics to automatically project upcoming monthly limits while warning of predictive threshold breaches.

Technical Infrastructure Stack

Frontend Stack: React.js (Vite Core Engine) styled natively using utility-first Tailwind CSS.	Backend Engine: Node.js framework initialized with Express.js server routing layers.
Database Layer: Relational structure mapped inside MySQL / Structured SQL environments.	Prediction System: Python Environment utilizing Scikit-Learn libraries processing <i>Linear Regression</i> / <i>ARIMA</i> models.

Day 02: Requirement Gathering Specification

Functional Requirements Architecture

- **User Authentication:** Multi-tenant system mapping secure registrations, state-managed credentials validation, and custom account environment profiles.
- **Transaction Management:** Highly reliable structured CRUD parameters handling income streams, multi-category tags, and ledger logs.
- **Target Budgeting Goals:** Micro-management limits configurable across custom categories with calendar-month scoping fields.

- **Predictive Budget Engine:** Machine learning predictive workflows analyzing past usage timelines to forecast downstream spending.
- **Automated Alert Triggers:** Dynamic visual indicators tracking category consumption indexes at 80% and 100% boundary thresholds.

Non-Functional Design Metrics

- **Security Standard:** Industry-standard cryptographic data encryption incorporating salted password hashes (bcrypt) and stateless session payloads over JSON Web Tokens (JWT).
- **Response Velocity:** Data queries, calculations, and visual graph layers must compile and mount layout components in less than *1.5 seconds*.
- **Device Universality:** Flexible viewports mapping fluid component restructuring across mobile screens, tablet devices, and standard high-resolution desktop terminals.

Day 03: User & Component Module Identification

The solution is broken down into structured independent services, separating business logic boundaries from interface layers:

SYSTEM MODULE DEPENDENCY MAPPING

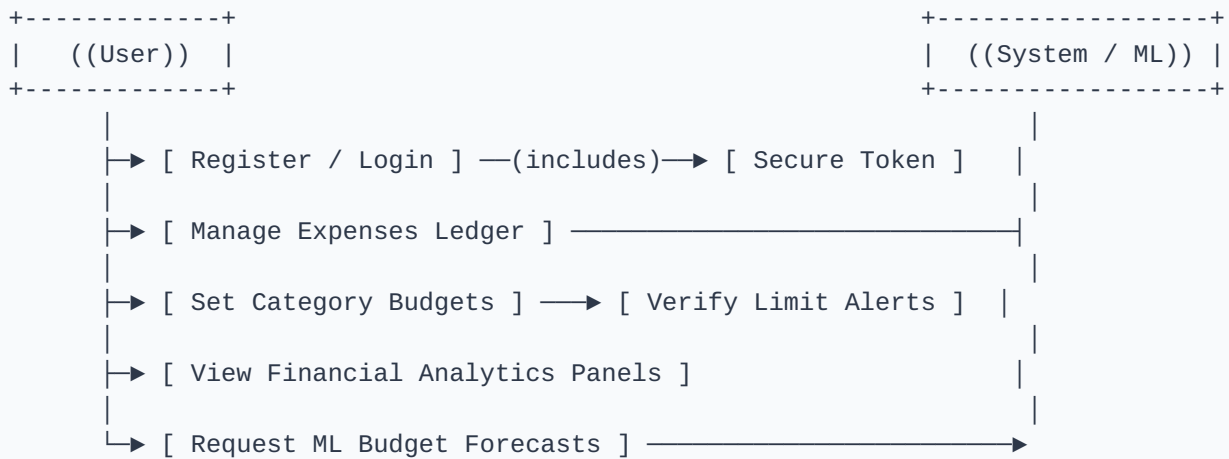
```
[📁 System Core Modules]
├── 👤 Auth Module      --> Coordinates structural sign-up, login, and token
    validations.
├── 💰 Transaction Core --> Manages inputs processing, numerical calculations, and
    logging.
├── 🚨 Alert Engine     --> Evaluates consumption states against user-defined limit
    rules.
├── 📊 Visualizer Stack --> Compiles raw tabular sets into data graphs via Chart.js
    modules.
└── 🧠 Prediction Unit --> Houses isolated machine learning calculations to process
    workflows.
```

Day 04: Use Case Interaction Modeling

The following mapping charts the system boundary interfaces between external actors and automated subsystem use cases:

SYSTEM USE CASE INTERFACE DIAGRAM

[USE CASE SYSTEM BOUNDARY]



Day 05: Data Requirement Blueprint Analysis

To establish data normalization, storage schemas are isolated into discrete transactional entities:

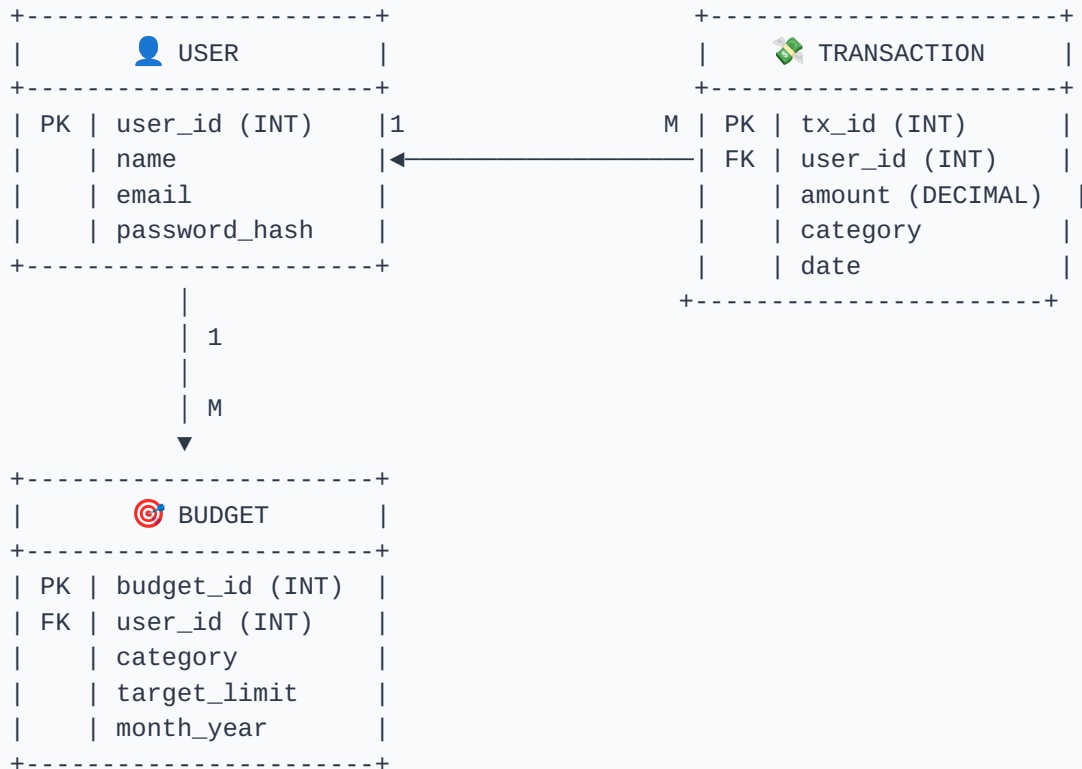
- **Users Entity:** System authorization attributes, security string hashes, primary default tracking currency settings, and creation tracking metadata.
- **Transactions Ledger:** Absolute transactional records compiling balance offsets, category grouping attributes, explicit transaction direction (income vs expense), date objects, and description fields.
- **Budgets Architecture:** Scoped baseline boundaries configuring maximum expected thresholds indexed against specific months and relational user nodes.
- **Machine Learning Forecast Store:** Optimization layer indexing predicted calculations to minimize server overhead and provide quick rendering.

Day 06: Entity-Relationship Structure Map

Logical relationships are mapped below, outlining the structural parent-to-child data distributions within the application:

SYSTEM CARDINALITY SCHEMA

[ENTITY RELATIONSHIP MAP]



Day 07: Relational Database Schema Build

The structured DDL SQL queries below set up the database infrastructure, relationships, and tracking keys:

```

-- Initialize User Accounts Relational Layout
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Initialize Ledger Records Processing Core
CREATE TABLE transactions (
    tx_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    amount DECIMAL(10, 2) NOT NULL,

```

```

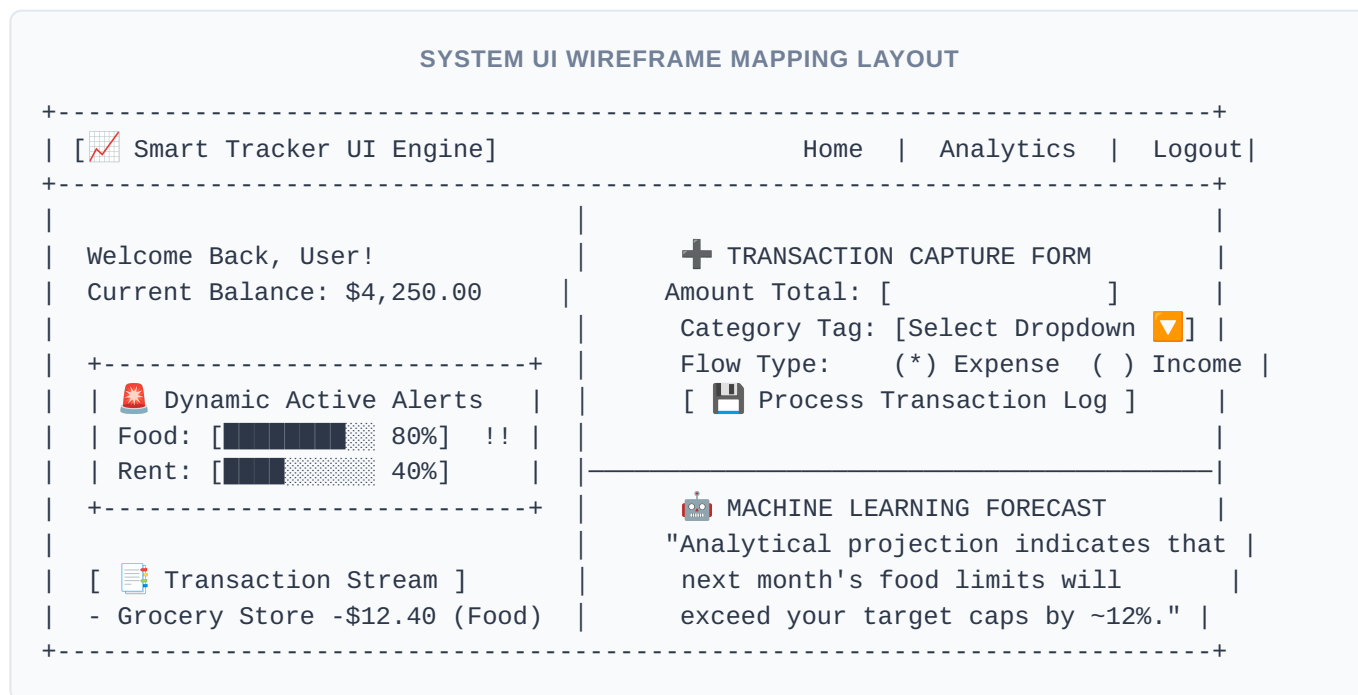
type ENUM('income', 'expense') NOT NULL,
category VARCHAR(50) NOT NULL,
date DATE NOT NULL,
description TEXT,
FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- Initialize Structural Budget Tracking Matrix
CREATE TABLE budgets (
    budget_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    category VARCHAR(50) NOT NULL,
    target_limit DECIMAL(10, 2) NOT NULL,
    month_year VARCHAR(7) NOT NULL, -- Logical Mapping Format: YYYY-MM
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

```

Day 08: Application Dashboard Wireframe Template

This layout outlines the responsive desktop dashboard wireframe pattern:



Day 09: Login & Dashboard View Architectures

- **Authentication Portal:** Centered multi-state access card featuring floating form validation layouts, async operational responses, native tracking filters, and sliding interface error boxes.
- **Main Dashboard System:** Grid system layout distributing analytical card frameworks (Total Credits, Net Outflows, Current Reserves), linked vector graph outputs via Chart.js, and quick entry fields.

Day 10: Routing Parameters & Usability Flows

- **Interface Navigation:** Persistent responsive navigation configuration utilizing application-wide state triggers. It fluidly shrinks into a collapsible navigation bar context for small touch-screen devices.
- **Smart Input Elements:** Contextual inputs featuring automated decimal masking, calendar inputs defaulting to system execution dates, and real-time frontend logic validation to handle incorrect entries before calling APIs.

Day 11: Design Quality & Structure Review

✓ Quality Verification Audit Status: PASSED

- **Accessibility Standards:** Color contrast configurations successfully clear standard WCAG AA guidelines, keeping screen layouts accessible and fully readable.
- **Relational Data Integrity:** System models verified to match Third Normal Form (**3NF**) constraints, eliminating redundant attributes and tracking loops.
- **Edge Exception Engineering:** Data entry pathways verified to stop overflow entries, malicious injection text, and data-type errors.

Day 12: Client Frontend Application Setup

Execute the following commands in the workspace terminal window to instantiate the complete React.js core environment layout:

```
# Initialize React Framework Core Workspace via Vite Engine
npm create vite@latest smart-expense-frontend -- --template react
cd smart-expense-frontend

# Inject Base Production Services and Layout Packages
npm install lucide-react chart.js react-chartjs-2 axios react-router-dom

# Configure CSS Compilation Framework (Tailwind Infrastructure)
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p

# Start the Local Development Framework
npm run dev
```